Ambient Systems Data

Data Segmentation

- **Firebase**: Ideal for real-time data, authentication, and scalable data storage. Use Firebase for:
  - Real-time updates and syncing.
  - User authentication and management.
  - Lightweight, non-relational data like logs or session data.
- **PostgreSQL**: Best for structured, relational data where complex queries, transactions, and data integrity are crucial. Use PostgreSQL for:
  - Storing detailed user data, financial records, and metrics.
  - Performing complex queries and aggregations.
  - Maintaining data relationships and ensuring data integrity.

Data Flow

- **User Input Forms**: Data entered through forms in your app can be submitted to PostgreSQL. Use APIs to handle form submissions and interact with PostgreSQL.
- **Real-Time Data Handling**: Firebase can handle real-time updates and events, but you'll push necessary data to PostgreSQL for detailed analysis and reporting.

Integration and Syncing

- **APIs**: Create APIs or use middleware to handle data synchronization between Firebase and PostgreSQL. For example:
  - Sync user authentication and session data between Firebase and PostgreSQL.
  - Transfer relevant data (e.g., user inputs, form submissions) from the app into PostgreSQL for processing and storage.
- **ETL Processes**: Use Extract, Transform, Load (ETL) processes to periodically or real-time sync data between Firebase and PostgreSQL. This ensures consistency and up-to-date information across systems

1. **Transportation**

- **Fuel Logs**
  - `id` (SERIAL PRIMARY KEY)
  - `user_id` (INTEGER)
  - `vehicle_id` (INTEGER)

- `fuel_type` (VARCHAR(50))
- `fuel_date` (DATE)
- `fuel_quantity` (NUMERIC(10, 2))
- `fuel_cost` (NUMERIC(10, 2))
- `odometer_reading` (NUMERIC(10, 2))
- `created_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
- `updated_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)

- **GPS Tracking Data**
  - `id` (SERIAL PRIMARY KEY)
  - `user_id` (INTEGER)
  - `vehicle_id` (INTEGER)
  - `tracking_date` (DATE)
  - `start_location` (POINT) -- Geospatial data (latitude, longitude)
  - `end_location` (POINT)
  - `distance_travelled` (NUMERIC(10, 2))
  - `average_speed` (NUMERIC(5, 2))
  - `route_taken` (TEXT)
  - `created_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
  - `updated_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
- **Vehicle Telematics Data**
  - `id` (SERIAL PRIMARY KEY)
  - `user_id` (INTEGER)
  - `vehicle_id` (INTEGER)
  - `telematics_date` (DATE)
  - `engine_performance` (TEXT)
  - `fuel_efficiency` (NUMERIC(5, 2))
  - `maintenance_alerts` (TEXT)
  - `driver_behavior` (TEXT)
  - `created_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
  - `updated_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
- **Fleet Management Systems**
  - `id` (SERIAL PRIMARY KEY)
  - `user_id` (INTEGER)
  - `fleet_id` (INTEGER)
  - `vehicle_ids` (INTEGER[]) -- Array of vehicle IDs

    - `maintenance_schedule` (TEXT)
    - `fuel_logs` (TEXT) -- Could store JSON data
    - `insurance_details` (TEXT)
    - `created_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
    - `updated_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
- **Delivery Logs**
    - `id` (SERIAL PRIMARY KEY)
    - `user_id` (INTEGER)
    - `delivery_date` (DATE)
    - `vehicle_id` (INTEGER)
    - `route_id` (VARCHAR(20))
    - `delivery_status` (VARCHAR(50))
    - `delivery_address` (TEXT)
    - `customer_name` (VARCHAR(255))
    - `delivery_time` (TIME)
    - `created_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
    - `updated_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)

    - `id` (SERIAL PRIMARY KEY)
    - `user_id` (INTEGER)
    - `vehicle_id` (INTEGER)
    - `maintenance_date` (DATE)
    - `maintenance_type` (VARCHAR(100))
    - `maintenance_cost` (NUMERIC(10, 2))
    - `maintenance_provider` (VARCHAR(255))
    - `next_maintenance_due` (DATE)
    - `created_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
    - `updated_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)

2. **Commercial Real Estate**

- **Financial Statements**
    - `id` (SERIAL PRIMARY KEY)
    - `user_id` (INTEGER)
    - `statement_type` (VARCHAR(50))
    - `period_start_date` (DATE)
    - `period_end_date` (DATE)

    - `total_income` (NUMERIC(15, 2))
    - `total_expenses` (NUMERIC(15, 2))
    - `net_profit` (NUMERIC(15, 2))
    - `total_assets` (NUMERIC(15, 2))
    - `total_liabilities` (NUMERIC(15, 2))
    - `created_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
    - `updated_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
- **Property Purchase Records**
    - `id` (SERIAL PRIMARY KEY)
    - `user_id` (INTEGER)
    - `property_id` (INTEGER)
    - `purchase_date` (DATE)
    - `purchase_price` (NUMERIC(15, 2))
    - `seller_name` (VARCHAR(255))
    - `property_location` (TEXT)
    - `property_type` (VARCHAR(100))
    - `mortgage_details` (TEXT)
    - `created_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
    - `updated_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
- **Sale Records**
    - `id` (SERIAL PRIMARY KEY)
    - `user_id` (INTEGER)
    - `property_id` (INTEGER)
    - `sale_date` (DATE)
    - `sale_price` (NUMERIC(15, 2))
    - `buyer_name` (VARCHAR(255))
    - `property_location` (TEXT)
    - `property_type` (VARCHAR(100))
    - `agent_details` (TEXT)
    - `created_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
    - `updated_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
- **Property Management Systems**
    - `id` (SERIAL PRIMARY KEY)
    - `user_id` (INTEGER)
    - `property_id` (INTEGER)

    - `tenant_name` (VARCHAR(255))
    - `lease_start_date` (DATE)
    - `lease_end_date` (DATE)
    - `monthly_rent` (NUMERIC(10, 2))
    - `maintenance_records` (TEXT) -- Could store JSON data
    - `payment_status` (VARCHAR(50))
    - `created_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
    - `updated_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
- **Property Valuation Reports**
    - `id` (SERIAL PRIMARY KEY)
    - `user_id` (INTEGER)
    - `property_id` (INTEGER)
    - `valuation_date` (DATE)
    - `valuation_amount` (NUMERIC(15, 2))
    - `valuer_name` (VARCHAR(255))
    - `valuation_method` (TEXT)
    - `property_location` (TEXT)
    - `created_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
    - `updated_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
- **Rental Income Records**
    - `id` (SERIAL PRIMARY KEY)
    - `user_id` (INTEGER)
    - `property_id` (INTEGER)
    - `rent_collection_date` (DATE)
    - `rent_amount` (NUMERIC(10, 2))
    - `tenant_name` (VARCHAR(255))
    - `payment_method` (VARCHAR(50))
    - `payment_status` (VARCHAR(50))
    - `created_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
    - `updated_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
- **Mortgage Documents**
    - `id` (SERIAL PRIMARY KEY)
    - `user_id` (INTEGER)
    - `property_id` (INTEGER)
    - `mortgage_lender` (VARCHAR(255))

- `loan_amount` (NUMERIC(15, 2))
- `interest_rate` (NUMERIC(5, 2))
- `loan_term` (INTEGER) -- Number of months or years
- `monthly_payment` (NUMERIC(10, 2))
- `start_date` (DATE)
- `end_date` (DATE)
- `created_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
- `updated_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)

- **Energy Bills**
  - `id` (SERIAL PRIMARY KEY)
  - `user_id` (INTEGER)
  - `property_id` (INTEGER)
  - `bill_date` (DATE)
  - `energy_consumed` (NUMERIC(10, 2))
  - `energy_cost` (NUMERIC(10, 2))
  - `provider_name` (VARCHAR(255))
  - `created_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
  - `updated_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)

3. **Vertical Farming**

- **Harvest Logs**
  - `id` (SERIAL PRIMARY KEY)
  - `user_id` (INTEGER)
  - `farm_id` (INTEGER)
  - `crop_type` (VARCHAR(100))
  - `harvest_date` (DATE)
  - `quantity_harvested` (NUMERIC(10, 2))
  - `market_price` (NUMERIC(10, 2))
  - `quality_grade` (VARCHAR(50))
  - `created_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
  - `updated_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
- **Farm Management Systems**
  - `id` (SERIAL PRIMARY KEY)
  - `user_id` (INTEGER)
  - `farm_id` (INTEGER)
  - `crop_planted` (VARCHAR(100))

- `planting

1. Manual Data Entry

- **User Interface (UI)**:
  - Build forms in the app's UI corresponding to the schema for each industry (e.g., fields like `date`, `vehicleId`, `routeId` for transportation logs).
  - Forms should include validation to ensure correct data entry (e.g., ensuring that `date` is in the correct format).
- **Backend Process**:
  - When the user submits the form, the app sends a request to the backend (e.g., using REST API or GraphQL).
  - The backend processes the request and inserts the data into the corresponding PostgreSQL tables.
- **Database Interaction**:
  - The backend can use an ORM (e.g., SQLAlchemy for Python) or raw SQL queries to interact with PostgreSQL.

2. File Upload

- **User Interface (UI)**:
  - Provide a file upload interface where users can upload CSV, Excel, or JSON files.
  - Allow users to map the columns in their file to the fields in your schema to ensure the correct data is inserted into the database.
- **Backend Process**:
  - Once the file is uploaded, the backend parses the file.
  - Implement validation checks to ensure the file contains the correct data format. For example, check that dates are valid and that required fields are present.
  - The parsed data is then inserted into the PostgreSQL database in bulk using `COPY` for CSV or batch inserts for other formats.
- **Database Interaction**:
  - Alternatively, use a Python script or ORM to read and insert the data into PostgreSQL.

3. Real-Time Database Sync

- **User Interface (UI)**:
  - Provide options for users to connect their existing databases through API keys, database credentials, or integration tools.
  - Allow users to configure the frequency and nature of data synchronization (e.g., real-time, hourly, daily).
- **Backend Process**:

    - Implement APIs that can connect to the users' real-time databases (e.g., PostgreSQL, MySQL, or others).
    - Use tools like Apache Kafka, Debezium, or (CDC) to listen for real-time changes in the users' databases and sync them to the Ambient Systems database.
    - Create a service that polls the external database for changes at a user-defined interval (e.g., every minute, hour).
- **Database Interaction**:
    - On detecting changes in the external database, insert/update the relevant data in the PostgreSQL database.
    - Use PostgreSQL's `INSERT ... ON CONFLICT` or UPSERT statements to handle new data or updates.

Additional Considerations:

- **Security**: Ensure that all data entry methods are secure by implementing authentication, encryption, and validation at every step.
- **Error Handling**: Provide clear error messages and logging to track issues during data entry.
- **Performance**: Use batch processing for large file uploads to improve performance. Optimize real-time sync mechanisms to avoid bottlenecks.
- **Scalability**: Ensure the app can handle growing amounts of data and concurrent user interactions, especially in the real-time sync method.

App Interaction Flow:

- **Manual Entry**: User → UI Form → Backend Validation → SQL Insert
- **File Upload**: User → UI Upload → File Parsing → Bulk Insert/Batch Insert
- **Real-Time Sync**: External DB → API/Kafka Listener → Sync Service → SQL Insert/Update

Validation Strategies for Each Method

1. **Manual Data Entry Validation**

- **Client-Side Validation (in the UI)**:
    - **Field Formats**: Validate that the data entered in the form fields matches the expected format (e.g., dates, times, numeric values).
        - Example: Ensure `date` fields are entered in `YYYY-MM-DD` format, and `vehicleId` fields are numeric.
    - **Required Fields**: Ensure that mandatory fields are filled out before the user can submit the form.
    - **Conditional Fields**: Validate that certain fields are completed based on the value of other fields (e.g., if `status` is "Delayed," a reason might be required).

    - **Feedback**: Provide immediate feedback to the user on any validation errors before form submission.
- **Server-Side Validation (in the backend)**:
    - **Data Integrity**: Perform additional checks to ensure data integrity before inserting it into the database.
        - Example: Ensure that `vehicleId` exists in a `vehicles` table, if such a reference exists.
    - **Business Logic**: Validate based on business rules (e.g., no overlapping schedules for the same `vehicleId`).
    - **Rate Limiting**: Prevent abuse by limiting the number of submissions per time period.

2. File Upload Validation

- **File Format Validation**:
    - **File Type**: Check that the uploaded file is in the correct format (e.g., CSV, Excel, JSON).
    - **Column Mapping**: Validate that the file contains all required columns and that the columns map correctly to the fields in your schema.
    - **File Size**: Ensure that the file size does not exceed a certain limit to avoid performance issues.
- **Data Validation**:
    - **Field Formats**: Ensure that the data within the file adheres to the expected formats.
    - **Range and Constraint Checks**: Validate that numerical values fall within acceptable ranges, dates are valid, and there are no invalid references.
    - **Duplicate Checks**: Ensure the file does not contain duplicate entries that could cause conflicts during the insert operation.
- **Bulk Processing Validation**:
    - **Batch Integrity**: Validate data in batches before insertion. If any data in the batch is invalid, reject the batch or log the errors for review.
    - **Error Reporting**: Provide a summary of any validation failures to the user after processing the file.

3. Real-Time Database Sync Validation

- **Connection Validation**:
    - **Credentials**: Validate that the database credentials provided by the user are correct and have the necessary permissions.
    - **Connection Stability**: Ensure that the connection to the external database is stable and reliable.
- **Data Validation**:
    - **Schema Compatibility**: Validate that the data from the external database is compatible with your app's schema.

    - **Conflict Resolution**: Handle conflicts between incoming data and existing data in the database (e.g., two updates happening simultaneously).
    - **Data Freshness**: Ensure that the data being synced is not outdated. This can be done by comparing timestamps.
- **Real-Time Sync Monitoring**:
    - **Error Logging**: Continuously monitor the sync process and log any errors or inconsistencies.
    - **Alerting**: Notify the user if there are persistent issues during synchronization, such as failed inserts due to validation errors.

Examples of Validation Rules

1. **Transportation Data (e.g., `delivery_logs`)**:
    - :
        - `date` : Must be a valid date in `YYYY-MM-DD` format.
        - `vehicleId` : Must be an integer and reference a valid vehicle.
        - `status` : Must be one of the allowed values (e.g., "Completed", "Delayed").
    - **Business Logic**: Ensure that delivery logs for the same vehicle on the same date do not overlap.
2. **Commercial Real Estate Data (e.g., `property_purchase`)**:
    - :
        - `purchasePrice` : Must be a positive numeric value.
        - `purchaseDate` : Must be a valid date, and it should not be a future date.
    - **Business Logic**: Validate that the purchase price aligns with industry standards or other reference data.
3. **Vertical Farming Data (e.g., `harvest_logs`)**:
    - :
        - `harvestAmount` : Must be a positive number representing the quantity of produce harvested.
        - `harvestDate` : Must be a valid date, and it should not be in the future.
    - **Business Logic**: Ensure that the harvest amount does not exceed the capacity of the farm plot.

Summary of Validation Layers

1. **Client-Side Validation**: Perform basic format checks, required fields validation, and conditional validation in the UI.
2. **Server-Side Validation**: Enforce business rules, data integrity, and security checks before inserting data into the database.
3. **Database-Level Validation**: Use constraints in PostgreSQL (e.g., foreign key constraints, unique constraints, check constraints) to enforce data consistency.

Industry
Data Source
Field Name
Field Format

All Industries	**Compliance Records**	`id`	SERIAL PRIMARY KEY
		`user_id`	INTEGER
		`compliance_type`	VARCHAR(100)
		`audit_date`	DATE
		`status`	VARCHAR(50)
		`compliance_officer`	VARCHAR(255)
		`comments`	TEXT
		`created_at`	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
		`updated_at`	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
	Customer Feedback	`id`	SERIAL PRIMARY KEY
		`user_id`	INTEGER
		`customer_name`	VARCHAR(255)
		`feedback_date`	DATE

		feedback_type	VARCHAR(50)
		rating	NUMERIC(3, 2)
		comments	TEXT
		created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
		updated_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
	Incident Reports	id	SERIAL PRIMARY KEY
		user_id	INTEGER
		incident_type	VARCHAR(100)
		incident_date	DATE
		location	VARCHAR(255)
		description	TEXT
		action_taken	TEXT
		created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
		updated_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP

		`user_id`	INTEGER
		`item_name`	VARCHAR(255)
		`item_type`	VARCHAR(100)
		`quantity`	NUMERIC(10, 2)
		`location`	VARCHAR(255)
		`reorder_level`	NUMERIC(10, 2)
		`supplier`	VARCHAR(255)
		`created_at`	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
		`updated_at`	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
Commercial Real Estate	**Tenant Records**	`id`	SERIAL PRIMARY KEY
		`user_id`	INTEGER
		`property_id`	INTEGER
		`tenant_name`	VARCHAR(255)
		`lease_start_date`	DATE
		`lease_end_date`	DATE
		`rent_amount`	NUMERIC(10, 2)
		`payment_status`	VARCHAR(50)

ion	VARCHAR(255)		
		created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
		updated_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
	Property Maintenance Schedules	id	SERIAL PRIMARY KEY
		user_id	INTEGER
		property_id	INTEGER
		maintenance_type	VARCHAR(100)
		schedule_date	DATE
		provider	VARCHAR(255)
		cost	NUMERIC(10, 2)
		status	VARCHAR(50)
		created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
		updated_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
	Leasing History	id	SERIAL PRIMARY

KEY			
		`user_id`	INTEGER
		`property_id`	INTEGER
		`tenant_name`	VARCHAR(255)
		`lease_start_date`	DATE
		`lease_end_date`	DATE
		`rent_amount`	NUMERIC(10, 2)
		`lease_terms`	TEXT
		`created_at`	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
		`updated_at`	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
	Market Analysis Reports	`id`	SERIAL PRIMARY KEY
		`user_id`	INTEGER
		`analysis_date`	DATE
		`market_segment`	VARCHAR(100)
		`property_type`	VARCHAR(100)
		`average_price`	NUMERIC(15, 2)
		`demand_trends`	TEXT

		`created_at`	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
		`updated_at`	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
Transportation	**Route Optimization Data**	`id`	SERIAL PRIMARY KEY
		`user_id`	INTEGER
		`vehicle_id`	INTEGER
		`route_id`	VARCHAR(20)
		`route_date`	DATE
		`start_location`	POINT
		`end_location`	POINT
		`optimized_route`	TEXT
		`travel_time`	INTERVAL
		`fuel_consumption`	NUMERIC(10, 2)
		`created_at`	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
		`updated_at`	TIMESTAMP DEFAULT CURRENT_TIMESTAMP

	Driver Performance Records	`id`	SERIAL PRIMARY KEY
		`user_id`	INTEGER
		`driver_id`	INTEGER
		`vehicle_id`	INTEGER
		`trip_date`	DATE
		`distance_travell`	, 2)
		`speeding_incidents`	INTEGER
		`fuel_efficiency`	NUMERIC(5, 2)
		`performance_rating`	NUMERIC(3, 2)
		`created_at`	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
		`updated_at`	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
	Accident Reports	`id`	SERIAL PRIMARY KEY
		`user_id`	INTEGER
		`vehicle_id`	INTEGER
		`driver_id`	INTEGER

		location	VARCHAR(255)
		cause	TEXT
		damage_estimate	NUMERIC(15, 2)
		police_report_number	VARCHAR(50)
		created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
		updated_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
	Insurance Claims	id	SERIAL PRIMARY KEY
		user_id	INTEGER
		vehicle_id	INTEGER
		claim_date	DATE
		claim_type	VARCHAR(50)
		claim_amount	NUMERIC(15, 2)
		status	VARCHAR(50)
		insurance_provider	VARCHAR(255)
		created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP

		`updated_at`	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
Vertical Farming	**Soil Health Logs**	`id`	SERIAL PRIMARY KEY
		`user_id`	INTEGER
		`farm_id`	INTEGER
		`test_date`	DATE
		`soil_type`	VARCHAR(100)
		`pH_level`	NUMERIC(5, 2)
		`nutrient_levels`	TEXT
		`recommendations`	TEXT
		`created_at`	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
		`updated_at`	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
	Pest and Disease Monitoring	`id`	SERIAL PRIMARY KEY
		`user_id`	INTEGER
		`farm_id`	INTEGER
		`crop_type`	VARCHAR(100)
		`monitoring_date`	DATE

		pest_disease_type	VARCHAR(100)
		severity	VARCHAR(50)
		treatment	TEXT
		created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
		updated_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
	Climate Control Data	id	SERIAL PRIMARY KEY
		user_id	INTEGER
		farm_id	INTEGER
		date_time	TIMESTAMP
		temperature	NUMERIC(5, 2)
		humidity	NUMERIC(5, 2)
		CO2_levels	NUMERIC(5, 2)
		light_intensity	NUMERIC(5, 2)
		created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP

	Water Usage Logs	`id`	SERIAL PRIMARY KEY
		`user_id`	INTEGER
		`farm_id`	INTEGER
		`irrigation_date`	DATE
		`water_source`	VARCHAR(255)
		`quantity_us`	, 2)
		`efficiency`	NUMERIC(5, 2)
		`created_at`	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
		`updated_at`	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
	Nutrient Solution Management	`id`	SERIAL PRIMARY KEY
		`user_id`	INTEGER
		`farm_id`	INTEGER
		`solution_date`	DATE
		`nutrient_formula`	TEXT
		`pH_level`	NUMERIC(5, 2)
		`EC_level`	NUMERIC(5, 2)

		`created_at`	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
		`updated_at`	TIMESTAMP DEFAULT CURRENT_TIMESTAMP